

系统资源实时预测实现方案

一、整体架构设计

二、计算或模型介绍

1、移动平均

2、基线计算

3、同比/环比增长率外推

4、单指数平滑计算

递推公式

参数建议

二、指标分类

1、周期性指标

(1) 检测方法

(2) 适配模型

2、平稳性指标

(1) 检测方法

(2) 适配计算

3、高波动性指标

(1) 检测方法

(2) 适配计算

三、落地推动

1、离线指标分类

2、实时计算

2.1 用户传参

2.2 pyflink读取

(1) 读取用户数据

(2) 读取数据库离线数据

2.3 pyflink计算

3、技术方案

3.1 技术架构设计

3.2 技术选型

3.3 实现步骤

3.4 模型训练与更新

3.5 完整代码示例

一、整体架构设计

采用：

- 指标分类
- 区别计算

二、计算或模型介绍

1、移动平均

- 适用场景：短期预测，数据波动较小且无明显趋势或季节性。
- 方案步骤：
 - 确定窗口期：例如用过去5期（分钟）的平均值预测下一期。
 - 计算移动平均值：

$$\blacksquare F_{t+1} = \frac{F_t + F_{t-1} + F_{t-2} + F_{t-3} + F_{t-4}}{5}$$

- 计算加权移动平均：

$$\blacksquare F_{t+1} = \frac{5F_t + 4F_{t-1} + 3F_{t-2} + 2F_{t-3} + F_{t-4}}{15}$$

2、基线计算

划分工作日、非工作日，过去一周或者两周时间段内，五分钟窗口的平均值。

3、同比/环比增长率外推

- 适用场景：数据稳定增长或下降，需快速估算未来值。

- 方案步骤：

- 计算增长率

- 同比增长率 = $\frac{Y_{今天t-5} - Y_{昨天t-5}}{Y_{昨天t-5}}$

- 环比增长率 = $\frac{Y_{今天t-5} - Y_{今天t-10}}{Y_{今天t-10}}$

- 外推公式

$$F_{t+1} = Y_t * (1 + \text{增长率})$$

4、单指数平滑计算

递推公式

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha) \hat{y}_t$$

$$\hat{y}_1 = y_1$$

其中,

$\hat{y}_{t+1}$, 下一时刻的预测值

y_t , 当前时刻的真实值

$\hat{y}_t$, 当前时刻的预测值

α , (平滑因子): 控制新数据权重 ($0 < \alpha \leq 1$)

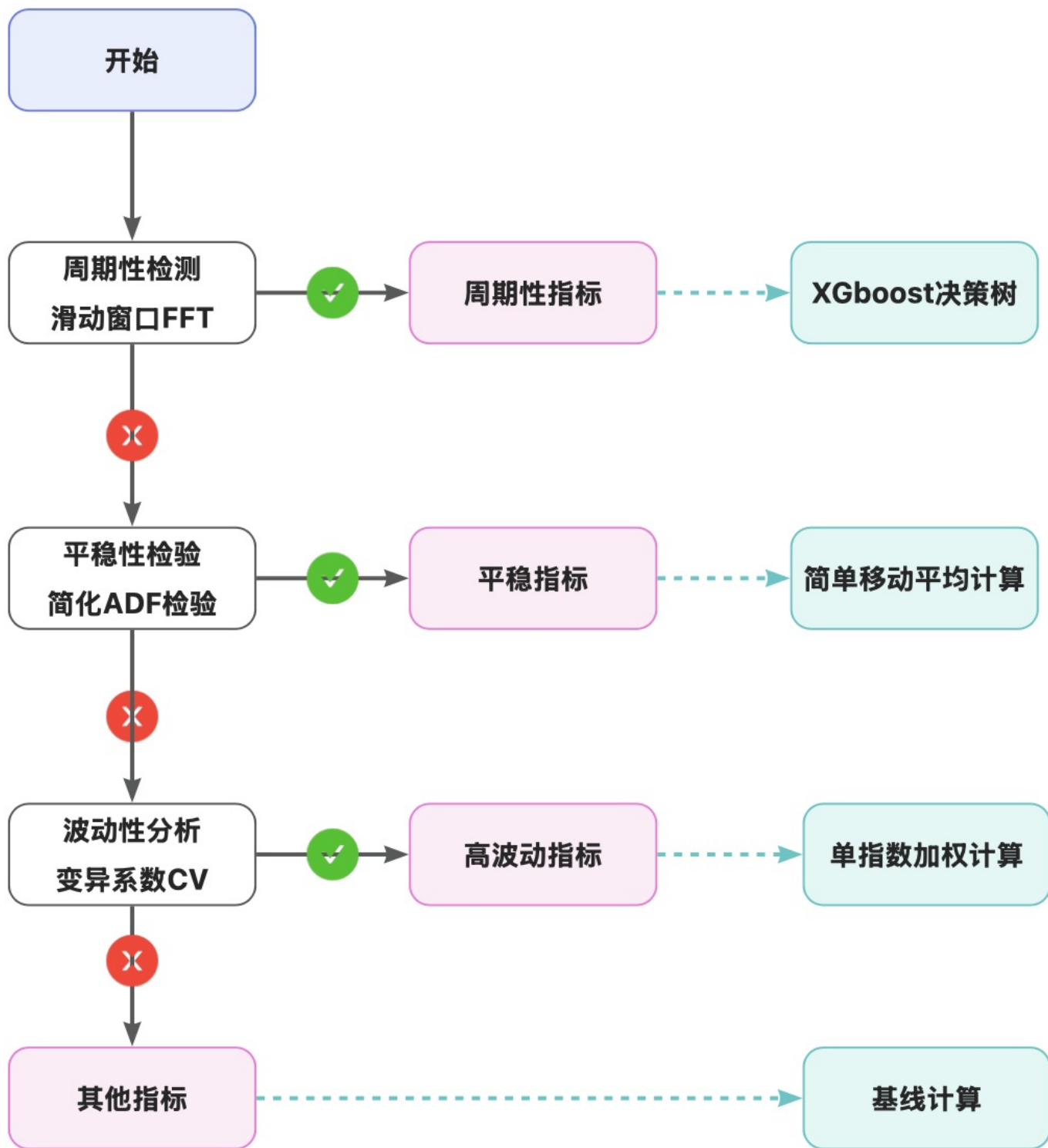
参数建议

$\alpha=0.5\sim0.7$ (高 α 赋予近期数据更高权重)

二、指标分类

流程步骤:

- 对历史指标数据进行分析, 离线提前打上标签;
- 根据不同标签的指标, 使用不同的实时预测策略。



1、周期性指标

(1) 检测方法

- 快速傅里叶变换（FFT）：
 - 实现：在滑动窗口内计算主频能量占比，若超过阈值（如60%），判定为周期性指标。

- 优化：使用增量FFT算法减少计算开销。

```
Python |  
1 def detect_periodicity(data_window, threshold=0.6):  
2     fft = np.fft.fft(data_window)  
3     power = np.abs(fft) ** 2  
4     main_freq_ratio = np.max(power) / np.sum(power)  
5     return main_freq_ratio > threshold
```

- 自相关函数（ACF）：

- 实现：检测时间序列自相关性的周期性峰值。
- 适用场景：数据噪声较高时，FFT可能失效，ACF可辅助验证。

(2) 适配模型

- **XGBoost模型**

核心参数调优指南：

- 控制复杂度：
 - max_depth（树的最大深度）
 - 作用：树越深，模型越复杂，推理越慢。
 - 3~6（浅树加快预测速度）。

```
Python |  
1 params = {'max_depth': 4} # 限制树深度
```

- n_estimators（树的数量）。
 - 树越多，模型越大，内存和计算成本越高。
 - 实时场景建议：50~200（结合早停法动态选择）

```
Python |  
1 # 启用早停法（当验证集指标连续10轮不提升时停止训练）  
2 model = xgb.train(params, dtrain, num_boost_round=1000, evals=watchlist, early_stopping_rounds=10)
```

- learning_rate（学习率，即 eta）
 - 学习率越小，需更多树拟合，但可能过拟合。

- 0.01~0.1（较低学习率需配合更多树，但需权衡速度）

```
1 params = {'eta': 0.05} # 平衡精度与树数量
```

- 正则化与抗过拟合
 - lambda（L2正则化）和 alpha（L1正则化），抑制叶子节点权重，防止过拟合

```
1 params = {'lambda': 2, 'alpha': 0.5} # 增强泛化能力
```

- subsample（行采样）和 colsample_bytree（列采样）
 - 随机采样数据/特征，提升鲁棒性。

```
1 params = {'subsample': 0.8, 'colsample_bytree': 0.8} # 行列采样
```

- 加速推理的工程参数
 - tree_method（树构造算法），hist（直方图算法）比 exact（精确贪心）更快且省内存。

```
1 params = {'tree_method': 'hist'} # 适合实时部署
```

2、平稳性指标

(1) 检测方法

- 简化ADF检验：
 - 实现：通过方差比（差分后方差/原始方差）替代完整ADF检验。

```
1 def check_stationarity(data_window, var_ratio_threshold=0.1):  
2     diff = np.diff(data_window)  
3     var_ratio = np.var(diff) / np.var(data_window)  
4     return var_ratio < var_ratio_threshold
```

- 滚动均值/方差检验：
 - 若窗口内均值和方差的波动小于阈值（如5%），判定为平稳。

(2) 适配计算

- 简单移动平均（SMA）：
 - 直接取窗口均值，计算成本极低。

3、高波动性指标

(1) 检测方法

- 变异系数（CV）
 - 实现： $CV = \text{标准差} / \text{均值}$ ，若超过阈值（如0.5），判定为高波动。

```
Python |  
1 def detect_high_volatility(data_window, cv_threshold=0.5):  
2     mean = np.mean(data_window)  
3     std = np.std(data_window)  
4     cv = std / mean if mean != 0 else 0  
5     return cv > cv_threshold
```

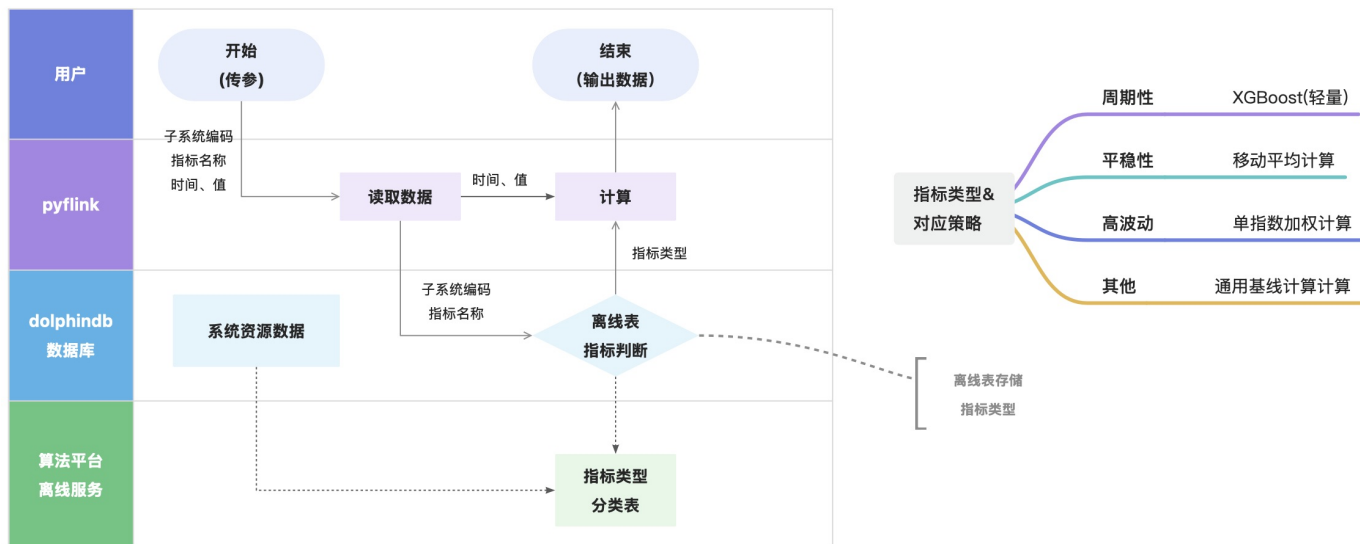
(2) 适配计算

指数加权移动平均（EMA）：

- 使用EMA快速响应变化

```
Python |  
1 def ema_zscore_predict(data_window, alpha=0.3, z_threshold=3.0):  
2     ema = data_window[0]  
3     residuals = []  
4     for value in data_window[1:]:  
5         ema = alpha * value + (1 - alpha) * ema  
6         residuals.append(value - ema)
```

三、落地推动



1、离线指标分类

系统创新部模型（张瑶玉）

- 支持T_1调度训练
- 自定义镜像（函数）

2、实时计算

数据研发部flink平台（马进伟）

pylink（郑志强）

2.1 用户传参

用户传入数据，包含以下字段：

- 子系统编码：sub_sys_code
- 指标类型：key_type
- 时间：date_time
- 值：value

2.2 pyflink读取

(1) 读取用户数据

- 识别参数子系统和指标类型
- 读取对应的时间和值

(2) 读取数据库离线数据

调用接口，读取dolphindb数据库的离线表，定位子系统和指标的类型

2.3 pyflink计算

- 针对不同类型指标，选择计算函数
- 计算后，输出预测值

3、技术方案

3.1 技术架构设计

Python

1 数据源（Prometheus/Kafka）→ PyFlink 流处理 → 特征工程 → 实时预测模型 → 结果输出（Redis/Kafka/可视化平台）

3.2 技术选型

组件	作用
PyFlink	流数据处理核心，支持窗口计算、状态管理和模型推理
Kafka	缓冲高吞吐数据流
TensorFlow/PyTorch	实时预测模型训练与推理（支持ONNX格式加速）
Redis	存储实时预测结果和模型版本信息

3.3 实现步骤

Step 1: 数据接入

Kafka Producer 将数据推送到主题

```
Python |  
1  from kafka import KafkaProducer  
2  import json  
3  
4  producer = KafkaProducer(  
5      bootstrap_servers='localhost:9092',  
6      value_serializer=lambda v: json.dumps(v).encode('utf-8')  
7  )  
8  
9  # 模拟 Prometheus 数据抓取  
10 while True:  
11     cpu_usage = get_cpu_usage() # 从 Prometheus 获取 CPU 使用率  
12     producer.send('system-metrics', {'timestamp': int(time.time()), 'cpu':  
        : cpu_usage})
```

Step 2: PyFlink 流处理

```
1 from pyflink.datastream import StreamExecutionEnvironment
2 from pyflink.table import StreamTableEnvironment, EnvironmentSettings
3 from pyflink.datastream.connectors import FlinkKafkaConsumer
4 from pyflink.common.serialization import SimpleStringSchema
5 import pandas as pd
6
7 # 创建流执行环境
8 env = StreamExecutionEnvironment.get_execution_environment()
9 t_env = StreamTableEnvironment.create(env, environment_settings=EnvironmentSettings.in_streaming_mode())
10
11 # 定义 Kafka 数据源
12 kafka_source = FlinkKafkaConsumer(
13     topics='system-metrics',
14     deserialization_schema=SimpleStringSchema(),
15     properties={'bootstrap.servers': 'localhost:9092'}
16 )
17
18 # 转换 JSON 字符串为 DataFrame
19 ds = env.add_source(kafka_source)
20 t_env.create_temporary_view("metrics", ds.map(lambda x: json.loads(x)))
21
22 # 注册窗口聚合查询（计算滚动平均）
23 query = """
24 SELECT
25     TUMBLE_START(timestamp, INTERVAL '1' MINUTE) AS window_start,
26     AVG(cpu) AS avg_cpu,
27     MAX(cpu) AS max_cpu,
28     COUNT(*) AS sample_count
29 FROM metrics
30 GROUP BY TUMBLE(timestamp, INTERVAL '1' MINUTE)
31 """
32
33 result_table = t_env.sql_query(query)
```

Step 3: 特征工程

关键特征：

- 时间窗口统计：滚动均值、最大值、样本数
- 外部特征：时间戳（小时、星期几）、历史预测误差
- 衍生特征：移动平均差分、波动率

```
1 # 添加时间特征
2 @udf(input_types=["TIMESTAMP"], result_type="INT")
3 def hour_of_day(ts):
4     return ts.hour
5
6 result_table = result_table.with_columns(hour_of_day(col("window_start")))
```

Step 4: 实时计算或模型预测

例如，模型部署方式：轻量级模型，直接嵌入 Flink 作业（XGBoost）

```
1 from pyflink.udf import udf
2 import joblib # 加载预训练模型
3
4 # 加载本地模型文件
5 model = joblib.load('cpu_predictor.pkl')
6
7 # 定义预测 UDF
8 @udf(input_types=["FLOAT", "INT", "FLOAT"], result_type="FLOAT")
9 def predict(cpu_avg, hour, sample_count):
10     input_features = [[cpu_avg, hour, sample_count]]
11     return model.predict(input_features)[0]
12
13 # 应用预测
14 result_table = result_table.with_columns(
15     predict(col("avg_cpu"), col("hour_of_day"), col("sample_count"))
16 )
```

Step 5: 结果输出与告警

写入 Redis 存储预测结果

```
1 from pyflink.datastream.connectors import FlinkJedisPoolSink
2
3 jedis_sink = FlinkJedisPoolSink.builder() \
4     .setHost("localhost") \
5     .setPort(6379) \
6     .setKeySerializer(StringSerializer()) \
7     .setValueSerializer(JsonRowSerializer(schema)) \
8     .build()
9
10 result_table.add_sink(jedis_sink)
```

告警规则（示例）

```
1 if predicted_cpu > threshold:
2     trigger_autoscaling(action="scale_out")
```

3.4 模型训练与更新

离线训练（每日凌晨执行）

```
1 # 使用历史数据训练模型
2 df = pd.read_parquet("historical_metrics.parquet")
3 model = RandomForestRegressor().fit(df[features], df["cpu"])
4 joblib.dump(model, "cpu_predictor.pkl")
```

3.5 完整代码示例

```
1 # system_monitor.py
2 from pyflink.datastream import StreamExecutionEnvironment
3 from pyflink.table import StreamTableEnvironment
4 import json
5 import joblib
6
7 def main():
8     env = StreamExecutionEnvironment.get_execution_environment()
9     t_env = StreamTableEnvironment.create(env)
10
11     # 1. 数据接入
12     kafka_source = FlinkKafkaConsumer(
13         'system-metrics',
14         SimpleStringSchema(),
15         {'bootstrap.servers': 'localhost:9092'}
16     )
17     ds = env.add_source(kafka_source)
18     t_env.create_temporary_view("metrics", ds.map(json.loads))
19
20     # 2. 特征工程
21     query = """
22     SELECT
23         TUMBLE_START(timestamp, INTERVAL '1' MINUTE) AS window_start,
24         AVG(cpu) AS avg_cpu,
25         HOUR(TUMBLE_START(timestamp, INTERVAL '1' MINUTE)) AS hour_of_day,
26         COUNT(*) AS sample_count
27     FROM metrics
28     GROUP BY TUMBLE(timestamp, INTERVAL '1' MINUTE)
29     """
30     result_table = t_env.sql_query(query)
31
32     # 3. 模型预测
33     model = joblib.load('cpu_predictor.pkl')
34     @udf(input_types=["FLOAT", "INT", "INT"], result_type="FLOAT")
35     def predict(cpu_avg, hour, count):
36         return model.predict([[cpu_avg, hour, count]])[0]
37
38     result_table = result_table.with_columns(predict(col("avg_cpu"), col(
39         "hour_of_day"), col("sample_count")))
40
41     # 4. 输出结果
42     result_table.execute_insert("redis-sink").wait()
43
44 if __name__ == "__main__":
45     main()
```

其中，cpu_predictor.pkl，保存了预训练机器学习模型的文件，通过 Python 的 pickle 或 joblib 库序列化生成。

在实时预测系统中，作用是为流处理任务（如 PyFlink 作业）提供已训练好的模型，用于对新数据进行实时预测。

生成文件

步骤示例（以 XGBoost 为例）

- 离线训练模型（使用历史数据）

```
1 import xgboost as xgb
2 import joblib
3
4 # 假设已有历史数据集（特征：时间、CPU使用率、内存使用率等）
5 X_train = [...] # 特征矩阵
6 y_train = [...] # 目标值（未来CPU使用率）
7
8 # 训练模型
9 model = xgb.XGBRegressor()
10 model.fit(X_train, y_train)
11
12 # 保存模型到文件
13 joblib.dump(model, "cpu_predictor.pkl")
```

- 在 PyFlink 中加载模型

```
1 # 在 Flink 作业中加载模型文件
2 model = joblib.load("cpu_predictor.pkl")
3
4 # 定义预测 UDF（用户自定义函数）
5 @udf(input_types=["FLOAT", "INT"], result_type="FLOAT")
6 def predict(cpu_usage, memory_usage):
7     input_features = [[cpu_usage, memory_usage]]
8     return model.predict(input_features)[0]
```

